WebHealth Cloud Architecture & Technical Verification

Implementation Report: Fully Decoupled Microsoft Azure Microservices Pipeline

1. Executive Summary & Verification Overview

WebHealth is an enterprise distributed dead link detection and website health monitoring platform deployed on **Microsoft Azure**. The architecture adheres strictly to cloud microservice principles: the API layer is fully decoupled from crawler execution and parallel worker verification. Independent **Azure Functions workers** pull tasks asynchronously from **Azure Service Bus**, execute multi-tier HTTP probes, and idempotently persist structured telemetry in **Azure Cosmos DB**.

2. Implemented Features & Technical Verifications

- **Independent Worker Component:** Worker logic extracted into a standalone deployable Azure Function (worker/) using Node.js v4 programming model with native Service Bus queue triggers.
- **Non-Blocking Job-Based API:** POST /api/scan creates the scan job in Cosmos DB, validates input, and returns HTTP 202 Accepted with scanId immediately. Crawling and task dispatch run in detached background execution.
- **Dual-Scope Link Detection:** Recursively crawls internal HTML pages (depth 1–4), while extracting and verifying both **internal** and **external** destination links.
- **Standardized Link Classification:** Enforces strict status categories: WORKING (2xx), REDIRECT (3xx with target URL tracking), BROKEN (4xx, 5xx, host connection errors), and TIMEOUT (10s threshold).
- **Cosmos DB Partitioning & Idempotency:** Container scans partitioned on /id; container links partitioned on /scanId. Link document IDs use deterministic SHA-256 hashing (sha256(scanId + url)) to prevent duplicate records during retries.
- **Atomic Completion Tracking:** Atomic counters for totalLinks and processedLinks; transitions state to COMPLETED when all links finish, dynamically computing health score and P50/P95 latency percentiles.
- **Application Insights & Observability:** Wired applicationinsights SDK with automatic request, dependency, and console collection, attaching custom dimensions (scanId, latencyMs) for Log Analytics.
- **SSRF Security Guard:** Added ssrfGuard.js validating all user URLs against private IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.1) and cloud metadata services (169.254.169.254).
- **Transparent Environment Modes:** Explicit high-visibility UI banner displaying **SANDBOX EMULATION MODE (OFFLINE)** vs. **LIVE MICROSOFT AZURE CLOUD (PRODUCTION)**.
- **Unified Infrastructure Script & CI/CD:** deploy_azure.ps1 provisions App Service, Function App, Service Bus with Dead-Letter Queue (DLQ=5), Cosmos DB, Application Insights, and Key Vault. Two independent GitHub Actions workflows deploy API and Worker.

3. Microsoft Azure Service Mapping

Component Role	Microsoft Azure Service	Architecture Implementation Details
API & Web Host	Azure App Service (Linux Node 20)	Hosts React SPA and Express REST API; validates SSRF guard and exposes /health liveness probe.
Asynchronous Queue	Azure Service Bus	Queue: dead-link-tasks. Max delivery count: 5; dead-lettering enabled for poisoned messages.
Parallel Workers	Azure Functions (Consumption)	Standalone serverless worker triggered by Service Bus; executes HEAD/GET probes and measures latency.
NoSQL Database	Azure Cosmos DB (NoSQL API)	Database: webhealth-db. Container scans (PK: /id) & links (PK: /scanId).

Monitoring & Logs	Application Insights & Log Analytics	Captures traces with custom dimension scanId; alert trigger on Dead-Letter Queue > 0.
Secrets Storage	Azure Key Vault	Stores Service Bus and Cosmos DB credentials securely via Key Vault reference syntax.

4. Multi-Cloud Parity Matrix

Role	AWS Equivalent	GCP Equivalent	Azure Implementation (WebHealth)
API / Entry	API Gateway / App Runner	Cloud Run	**Azure App Service**
Message Queue	Amazon SQS (DLQ)	Cloud Pub/Sub	**Azure Service Bus (Queue + DLQ)**
Serverless Workers	AWS Lambda (SQS Trigger)	Cloud Run Worker	**Azure Functions (Service Bus Trigger)**
Document Database	Amazon DynamoDB	Cloud Firestore	**Azure Cosmos DB (NoSQL API)**
Observability	CloudWatch Logs & Metrics	Cloud Logging	**Application Insights & Log Analytics**

5. End-to-End Processing Flow

1. **Client Submission & SSRF Validation:** React frontend submits target URL. The API runs validateTargetUrl() blocking loopback and private subnets.
2. **Job Ingestion:** Scan document created in Cosmos DB with INITIALIZING. HTTP 202 is returned immediately.
3. **Background Crawler:** Cheerio parses internal links up to max depth, normalizes URLs, and sets totalLinks.
4. **Batch Enqueueing:** Discovered links are submitted to Azure Service Bus queue dead-link-tasks in batches.
5. **Functions Worker Execution:** Azure Function instances trigger on queue messages, execute HTTP HEAD/GET probes, and measure latency.
6. **Idempotent Upsert & Atomic Counter:** Worker upserts link record with id = sha256(scanId + url) into Cosmos DB and increments processedLinks.
7. **Completion & Reporting:** When processedLinks == totalLinks, status switches to COMPLETED. Frontend polling renders score, latency charts, and CSV/JSON export.